

Frontend

What I built the lesson and the bake sale out of, where the SVG art direction broke down and what I did about it, and why the lesson is not gamified while the game is.

BRYCE WALTERS FRONTEND.PDF SEPTEMBER 2026

Starting point

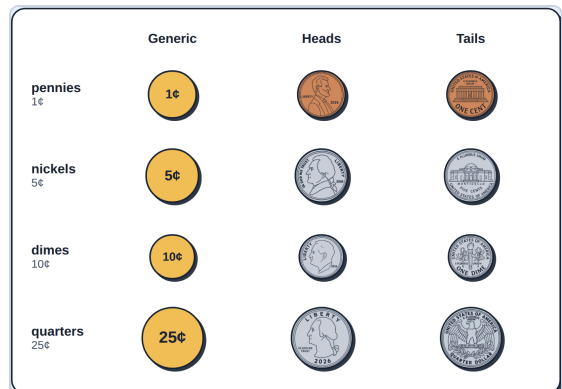
I used Vanilla JS to avoid relying too much on existing frameworks. Most of my experience in the last few years has been on an in-house Vanilla JS framework for building games rendered with SVG elements, so the general direction here is familiar, though almost none of it is carried over directly. Vite bundles and serves the app and does nothing else, and there are no runtime dependencies.

Componentizing first

I started frontend development by specifying what components comprise the lesson. The coins, the grid, the skip count grid and the narration box were all built as components before there was a screen to put them in, with particular attention on building individual testers for the coins and the skip count grid. I had three intentions in working that way:

1. Identification and development of re-usable deliverable game elements that can be used across lessons and games, to create a cohesive learning environment in the software.
2. A demonstration of basic coding principles — modules, and class inheritance. `Grid` is re-usable on its own; `SkipCountGrid` is an extension of it that could be used at an introductory skip count lesson; `SkipCountCurrencyGrid` is another level down, with overrides for the specific use in the lesson I built.
3. Optimization of Claude's context. For highly reusable and complex components it seems best to generate bespoke `CLAUDE.md` files, so individual elements can be updated without loading needless context. I have burned far too many tokens on personal projects by not splitting components into smaller independent contexts, so I was proactive about it here.

In practice the coin is the only one that has crossed over so far — the lesson's pile, the ghosted indicator in the chart and the wallet on the bake sale floor are all the same component, sized by one custom property so each coin keeps its real diameter against a quarter. The rest are built to be reused as work continues across the knowledge graph, which is an intention rather than a result.



The coin tester. Three display types across four denominations. The testers were built for quick human validation; in validating its own code Claude added regression checks to those pages, which I let stand.

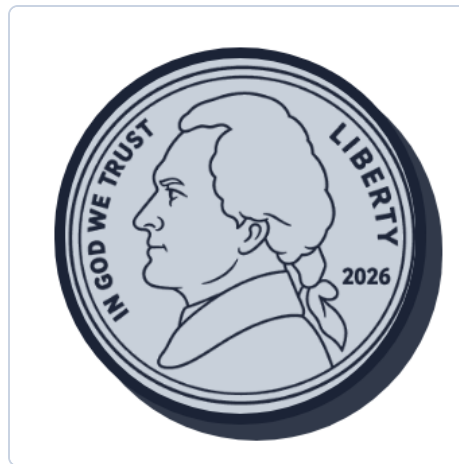
SVG, and where it broke down

My previous work landed largely in building with SVG graphics, which works well for scalable clean art, and some game elements can take advantage of SVG directly — building out animations with path elements, for instance. So I moved forward with that here, and the grids, the belt and the trays are all drawn that way.

The art itself is where it broke down. The nickel below is Claude's third attempt at simplified line art of a nickel head, given a reference image, and I tried every other coin side as well. The limitation is organic lines: it does well with clear geometry and single curves, but seems to fail when generating paths.



Generated as SVG — third attempt



Shipped — raster art in an SVG frame

Same coin, same component. The face on the right is a PNG delivered through an `<image>` inside the coin's SVG.

Rather than spend too much time on it — more than an hour — I tried ChatGPT and Gemini for the art, and found that ChatGPT was actually disregarding the SVG directive and building out PNGs anyway. The raster came out much more reliably than anything I got as paths. Ultimately I opted to use the `<image>` tag, so I could use the assets I had already generated and maintain the SVG framework I built up initially. The PNGs came from ChatGPT and Claude cleaned them up and sized them to fit the existing coin component.

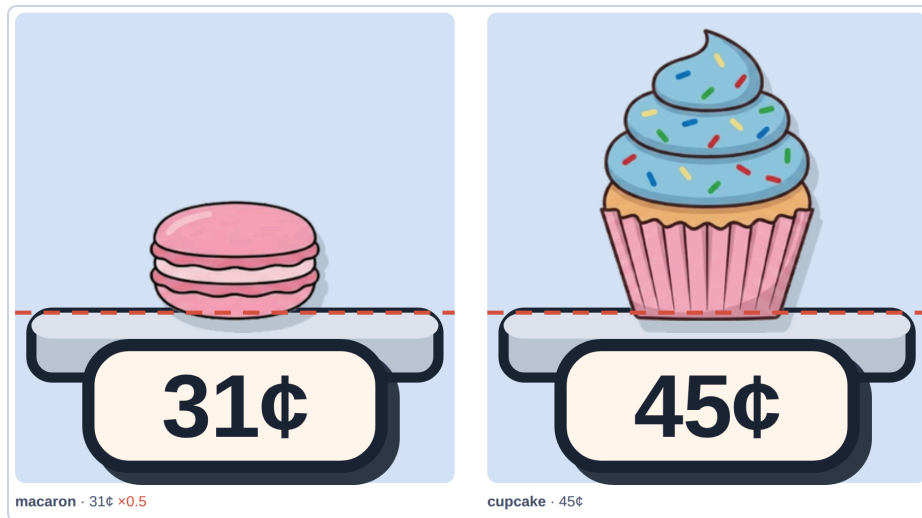
The faces are line art on a transparent ground, so the colour a child sees through them is the disc's own metal underneath. One set of files gives a copper penny and a silver dime, and swapping the set for drawn SVG later is a change in one module.

The mockup pass

I did most of the bake sale work in Claude's Cowork utility, composing it as a mockup out of existing components while folding in new ideas and placeholder art. The first pass had no bakery item assets at all, just boxes, which is the right way round — the frame is easier to judge before the art fills it.

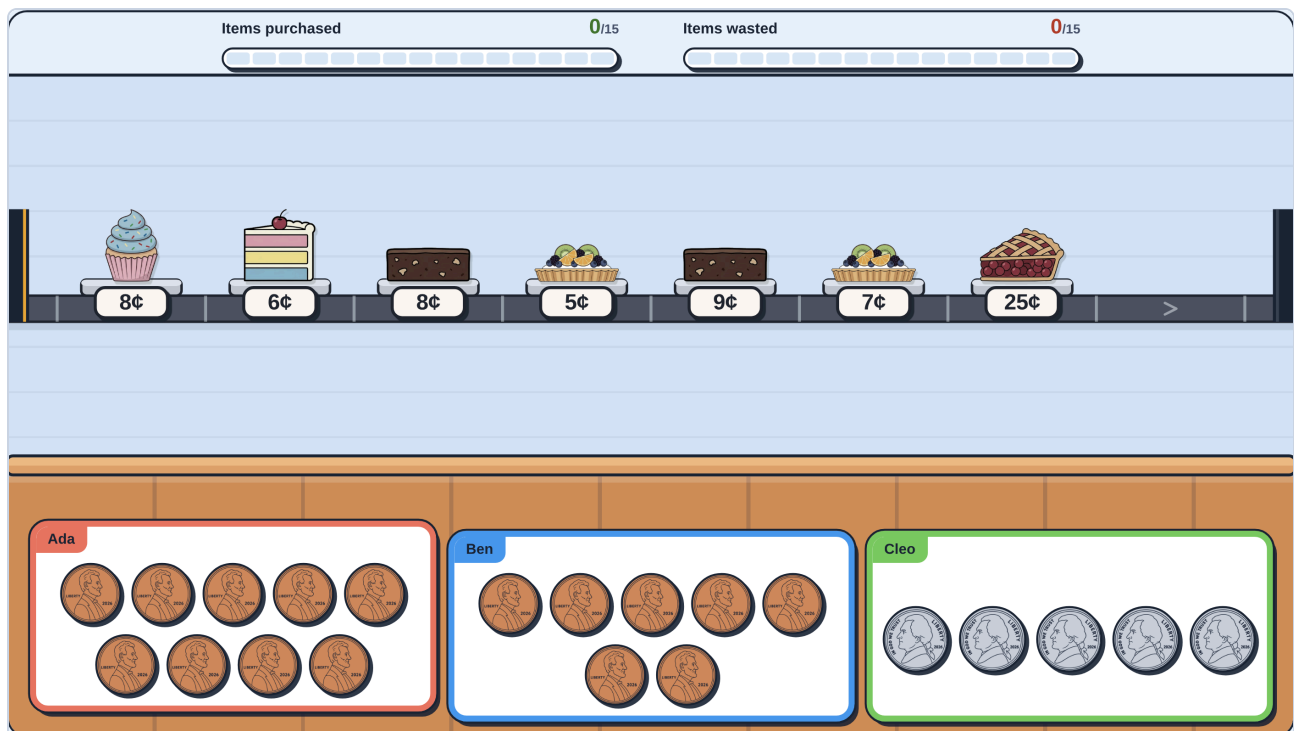
The decisions in that pass were mine. No cash register and no glass case on the counter, because they took up too much space. Ducts pulled back to a twenty pixel hard-edged slot rather than the chunky housings the first pass drew, with the belt running behind them so it feeds through rather than stopping at them. The supplied reference art punched out and used directly instead of drawn goods. The three CSS-drawn cake variants rejected. The macaron scaled down fifty per cent on the tray, where it read too large. The belt kept in SVG with its `viewBox` widened past the frame, rather than rewritten as DOM, so a newly baked tray slides in from off-stage instead of fading in where it stands. The bought chip cut. The coins left alone entirely, since they are

shared with the lesson where reading the coin is the exercise, and the app's cool ink kept rather than the mockup's warmer one. Two calls in that pass were Claude's proposals that I took: the meters as countable cells rather than a filling bar, and the rule that the food is warm and the machinery cool.



The tester for the goods. The dashed rule is the deck surface, drawn at 4× so it is obvious when a good does not stand on it.

The pass underscored that Claude is not good with SVG — that is the point at which I rejected the cake variants. It also underscored the importance of naming conventions for making concise changes to a mockup, and it proved Claude's competency at compositing images to generate a meaningful mockup. It revealed some spacing that I then did not have to discover during actual development. Applying the resulting markdown with Claude Code was seamless, and left little room for Claude to veer from the individual prompts I laid out.



The bake sale, built from that mockup. A belt of priced trays runs duct to duct above the counter; below it, three wallets whose value each player has to work out for themselves.

Gamification

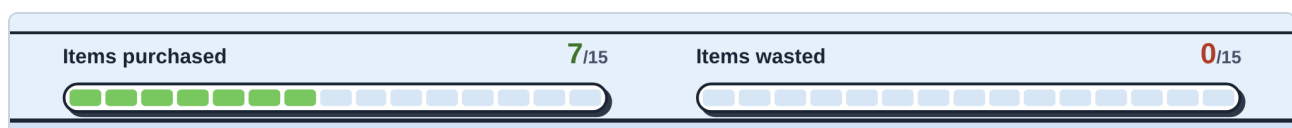
I left the lesson as a clean learning UI without any gamification, for two reasons. The first is that I think it is helpful for some experiences to focus solely on the math, and to give a break from the additional stimuli brought on by elements that exist only for gamification. Here I wanted a clean, deliberate UI that taught the concept and tracked how the student performed at it. The second is that having a variety of experiences, from a standard lesson through to a full game, would make me more interested as a student — a standard lesson for a clear learning experience, followed by a fun game for exercising that lesson, seems like an ideal flow.

One could have the coins slide into the side of the skip count grid like a coin-operated machine, filling it up based on their value. It is a good animation and it makes the mapping physical, but it sits on the one screen where that mapping is the lesson, so it can be added later without changing anything underneath it.

The multiplayer game

For the multiplayer game I wanted something loosely cooperative. Competitive games are really good for students that are closely matched in competence, and uniquely unrewarding when matched with a massive skill disparity. Cooperative games have the opportunity to pull out a desire for students to help one another succeed, which is something I have seen in classes first hand. A student with a high level of skill counting coins can look at another player's wallet and help them choose their value, over a voice chat ostensibly, and that is the interaction I want the game to produce.

The pitfall is students who quarterback the session, dictating every action and allowing the lower level student to sit back and not think critically. This game does not have a direct answer for that, but it does disallow students from taking direct action for each other, and a student who focuses too much on controlling the action runs the risk of losing sight of their personal involvement as well. The two meters are shared, so waste is collective.



The shared score. Items purchased and items wasted, as cells a child can count.

The game can scale difficulty by introducing a faster conveyor belt, and by reducing the forgiveness on the number of wasted food items. Neither needs new art or new UI.

Architecture

This is a fairly vanilla JS approach, using SVG largely as the UI elements on screen. Beyond the component and inheritance work above, the rest of the structure is conventional and worth stating briefly.

- A hash router owns the view lifecycle: it destroys the outgoing view before mounting the incoming one, so a view cannot leak listeners or timers into the next screen. Route guards live in the route table rather than inside views, so hosting a bake sale can be gated on finishing the lesson and a typed URL cannot walk around it. Joining someone else's room by code is deliberately ungated.
- One small observable store. Only three keys persist, under a versioned `localStorage` key that resets to defaults rather than throwing if the shape is wrong. Everything else is in memory.

- The server is authoritative for the bake sale. The client forwards what its player did and draws what it is told back; a hand's value is never sent to any client, because that sum is the exercise.
- One stylesheet per component, scoped by a root class, with colour, spacing and type coming from a tokens file. No `innerHTML` anywhere, since room codes and player names come off the wire.

Accessibility here is Claude's initiative rather than mine, and I let it stand: real buttons, `aria-pressed` on counted coins, nothing carried by colour alone, reduced motion honoured, and a caption under every narrated line. Accessibility is important to me, but for the context of this assignment I deprioritized it.

Sizing

I made the decision to let both the game and the lesson resize, and deliberately kicked off a refactor so the lesson elements would stay on screen at a reasonably sized display. Each screen is laid out once against a fixed design box and then scaled to fit, so the hundred chart's cells do not drift with the viewport — a chart you point at has to hold still. They are not designed for vertical screens or overly small screens.

Back to menu
Dev: mark lesson complete
Instruction

Press the Check Answer button to check.

Say it again
Sound on



is worth

15¢

Check Answer

1¢	2¢	3¢	4¢	5¢	6¢	7¢	8¢	9¢	10¢
11¢	12¢	13¢	14¢	15¢	16¢	17¢	18¢	19¢	20¢
21¢	22¢	23¢	24¢	25¢	26¢	27¢	28¢	29¢	30¢
31¢	32¢	33¢	34¢	35¢	36¢	37¢	38¢	39¢	40¢
41¢	42¢	43¢	44¢	45¢	46¢	47¢	48¢	49¢	50¢
51¢	52¢	53¢	54¢	55¢	56¢	57¢	58¢	59¢	60¢
61¢	62¢	63¢	64¢	65¢	66¢	67¢	68¢	69¢	70¢
71¢	72¢	73¢	74¢	75¢	76¢	77¢	78¢	79¢	80¢
81¢	82¢	83¢	84¢	85¢	86¢	87¢	88¢	89¢	90¢
91¢	92¢	93¢	94¢	95¢	96¢	97¢	98¢	99¢	100¢

The lesson, at the end of a demonstration. The pile, the chart with a real coin ghosted at each interval, and the typed answer. Every offset on the screen is a design pixel in a 1280×692 box.